

Documento Oficial do Projeto

Atualizador Inteligente de ERP

Arquitetura, estado de implementação e auditoria técnica contra o código e os binários reais de produção

Versão do documento: 1.0

Data: 27 de agosto de 2026

Autor da auditoria: Antonio Salomão

1. Visão geral do projeto

O Atualizador Inteligente de ERP é um ecossistema para eliminar a intervenção manual (hoje feita via AnyDesk, servidor por servidor) no processo de atualização de versões do ERP instalado nos clientes da Bredas Sistemas. Uma atualização pode alterar o banco de dados, executar scripts, substituir executáveis e afetar múltiplos terminais — por isso o processo precisa ser preparado, autorizado, executado e registrado de forma previsível.

O problema que o projeto resolve:

- O suporte precisa acompanhar cada atualização manualmente, cliente por cliente.
- Existe risco de esquecer o backup antes de aplicar alterações de banco.
- O cliente pode ser interrompido durante uma operação comercial.
- Pacotes em formatos diferentes exigem tratamento operacional adicional.
- É difícil saber rapidamente quais clientes foram atualizados, quais falharam e em qual etapa.

1.1 Arquitetura — três pilares

Pilar	Tecnologia	Responsabilidade
A Mente (API central)	Node.js, Express, SQLite	Gerenciar versões, pacotes, clientes, publicação, autenticação web e logs dos agentes.
O Músculo (Agente local)	C# / .NET 8 Worker Service	Operar no servidor do cliente: download, extração, GFIX, GBAK, BScript e atualização do BEXE.
O Rosto (ERP e terminais)	Delphi	Ler o estado da atualização, avisar o usuário e registrar a autorização no momento adequado.

1.2 Ciclo completo da atualização

Fase	O que acontece	Estado gravado
1 — Preparo invisível	Agente consulta a API (CNPJ + versão atual), baixa pacotes, confere SHA-256, extrai com 7za.exe.	PENDENTE
2 — Decisão do usuário	Delphi lê PENDENTE ao abrir o ERP, avisa o usuário; se confirmado, grava AUTORIZADO e se fecha.	AUTORIZADO
3 — Execução segura	gfix -shut force_0 → gbak (backup pré) → BScript.exe → gbak (backup pós) → gfix -online.	PROCESSANDO
4 — Distribuição limpa	Agente injeta os executáveis como BLOB no BEXE.fdb; terminais leem e se auto-atualizam.	CONCLUIDO / ERRO

Este ciclo elimina a antiga sequência manual de renomear/copiar o banco (Junior_.fdb → cópia → rename de volta): o isolamento via gfix -shut substitui essa dança inteira com uma técnica nativa do Firebird.

2. Escopo desta avaliação

Esta avaliação cruzou três documentos de arquitetura com o código-fonte real do projeto e, numa segunda passada, com os próprios binários e dados de produção — para separar o que já está construído e correto do que ainda é promessa de papel.

Artefato	Natureza	Cobertura
Planejamento_Tecnico_Atualizador_ERP_v4.pdf	Especificação técnica (ago/2026)	Arquitetura completa proposta — MD5, sem estado PROCESSANDO
Projeto_Arquitetura_Atualizador_ERP.pdf	Apresentação executiva	Mesma arquitetura da V4, em linguagem de negócio
Documento_Tecnico_Atualizador_ERP.pdf	Documento técnico v1.0 (26/ago/2026)	Versão revisada — SHA-256, estado PROCESSANDO, lista as próprias pendências
web/server, web/client	Código real — Node.js + Express + SQLite	Painel, API de versões/pacotes, autenticação, distribuição
Atualizador automatico/	Código real — C# / .NET 8 Worker Service	Agente local: polling, download, extração, GFIX/GBAK/BScript, injeção BEXE
Arquivos para analise/	Binários e dados reais de produção	BScript.exe e b_Atualizador.exe (extração de strings), BEXE.FDB real de 44 MB, BScript.Ini real, 1026 scripts DDL de 2011 a 12/08/2026

2.1 Resultado em números

Indicador	Valor
Comportamentos confirmados no código, batendo com o documento técnico v1.0	14
Achados críticos	2
Achados de alta severidade	4
Achados de média severidade	3
Achados de baixa severidade	3
Scripts DDL reais inspecionados (nenhum com proteção de idempotência própria)	1026

3. Comportamentos confirmados no código

Cada item abaixo é um comportamento que os documentos prometem e que o código realmente faz — verificado lendo a implementação, não a descrição dela.

- **Token do agente é exigido e comparado** — toda rota de polling/download/log exige cabeçalho X-Agent-Token idêntico ao configurado; sem token, 401.
- **SHA-256 de ponta a ponta** — painel calcula o hash no upload, agente recalcula após o download e aborta se não bater; nenhum resquício de MD5.
- **Nome de arquivo não escapa da pasta** — download usa path.basename; agente rejeita pacote cujo nome não seja idêntico ao declarado.
- **Sessão de login endurecida** — cookie HttpOnly, SameSite=lax, secure ligado a SESSION_SECURE, mais limitador de tentativas por IP.
- **Senha nunca aparece em log** — o executor de processos só registra o nome do binário chamado, nunca os argumentos.
- **Sequência crítica na ordem certa** — gfix -shut → gbak pré → BScript → gbak pós → injeção BEXE → gfix -online, exatamente como documentado.
- **Tentativa de rollback existe** — em qualquer exceção do fluxo crítico, tenta restaurar o backup pré-atualização e recolocar o banco online antes de marcar ERRO.
- **Nada de credencial fixa no código** — caminhos de GFIX/GBAK/BScript, credenciais Firebird, CNPJ, URL da API e token vêm todos de variável de ambiente.

4. Achados técnicos

Nenhum dos três documentos de arquitetura tem como revelar estes pontos — eles vivem em decisões de implementação que só aparecem lendo o C# e o Node.js linha a linha, ou, nos quatro primeiros casos, abrindo os próprios binários e a base de produção.

4.1 Contra os binários e a base reais

CRÍTICO O BScript.exe real parece ser um programa gráfico interativo — sem evidência de nenhum modo silencioso

A extração de strings do binário real (5 MB, compilado em 2019) revela um formulário Delphi/FireDAC completo: campos de texto para Servidor, Banco e pasta de Scripts; um seletor de arquivos .sql/.fdb com multisseleção manual; um RadioGroup para filtrar resultados; e dois botões — "Selecionar Scripts Pasta" e "Inserir No Banco Como Executado" — este último claramente uma confirmação manual, feita depois de olhar o resultado na tela.

Em nenhum lugar dos 5 MB do binário aparece uma string reconhecível de parsing de linha de comando: nenhuma ocorrência de /silent, -silent, FindCmdLineSwitch, -db= ou equivalente.

Onde: Arquivos para análise/BScript.exe (strings nos offsets ~5.08M e ~2.55M) · assumido em Worker.cs:79

Cenário: Worker.cs chama BScript.exe /silent /db="..." como se fosse linha de comando. Rodando num Windows Service (sessão 0, sem desktop interativo), o mais provável é uma janela que ninguém consegue ver nem clicar — e Process.WaitForExitAsync() fica esperando indefinidamente, com o banco em gfix -shut force_0 (bloqueado para todos) até alguém notar e matar o processo manualmente no servidor do cliente.

Correção: Confirmar com quem mantém o BScript.exe se existe versão/branch com modo linha de comando de verdade. Se não existir, precisa ganhar modo headless (ou ser substituído) antes de qualquer piloto.

ALTO O schema real do BEXE.fdb usa nomes diferentes dos que o agente assume

Abrindo o BEXE.FDB real de produção (44 MB) e extraíndo os identificadores das páginas de metadados do Firebird, a única tabela de aplicação encontrada é EXECUTAVEIS, com os campos NOMEPRODUTO, NOMEARQUIVO, VERSAO, VERSAOATUALIZADA, EXECUTAVEL, HASHEXE e DATA_ATUALIZACAO. O código do agente assume uma tabela VERSOES_EXE com colunas NOME_EXE e ARQUIVO_BLOB — nomes que não existem neste banco.

De bônus: o schema real já reserva um campo HASHEXE, provavelmente usado pelos terminais para validar o executável antes de rodar. O código atual não escreve nada nele.

Onde: DatabaseService.cs:62-64 (nomes assumidos) vs. Arquivos para análise/BEXE.FDB (schema real)

Cenário: Contra o arquivo real, InjetarNovosBinarios falharia já na primeira execução.

Correção: Reescrever InjetarNovosBinarios contra o schema real (EXECUTAVEIS / NOMEARQUIVO / EXECUTAVEL / HASHEXE) e decidir o que gravar em HASHEXE — o SHA-256 já calculado no download é o candidato natural.

MÉDIO Porta do Firebird divergente entre a configuração real e o agente

O BScript.Ini real, em uso hoje, aponta Server=localhost/3051 — uma porta não padrão. O agente C# grava Port=3050 fixo na connection string, sem variável de ambiente para ajustar.

Onde: DatabaseService.cs:12 vs. Arquivos para análise/BScript.Ini:2

Cenário: Num cliente cujo Firebird escute em 3051 (como este arquivo de configuração real evidencia que acontece), o agente simplesmente não conseguiria abrir conexão nenhuma com o JUNIOR.fdb.

Correção: Expor a porta como variável de ambiente (ex.: ATUALIZADOR_DB_PORT), com 3050 como padrão.

BAIXO 1026 scripts DDL reais, nenhum protegido contra reexecução — e "já aplicado" hoje é um clique manual

Amostrando os scripts em Scripts-BVendas/ (1026 arquivos, de 2011 até 12/08/2026), nenhum tem checagem de existência prévia (nem IF NOT EXISTS, nem EXECUTE BLOCK consultando RDB\$RELATION_FIELDS). No próprio BScript.exe, o botão "Inserir No Banco Como Executado" é uma ação manual, separada da execução em si.

Isso dá corpo ao risco de idempotência: sem marcação automática, rodar os scripts sem saber quais já foram aplicados naquele cliente gera erros "objeto já existe" em qualquer cliente que não parta de um banco zerado — exatamente os erros que a política atual do agente (qualquer código de saída ≠ 0 aborta e reverte) trataria como falha crítica.

Onde: Scripts-BVendas/*.sql (amostra) · fluxo de dois botões do BScript.exe · ProcessService.cs:35-36

Correção: O BScript precisa de uma tabela de controle que ele mesmo consulte antes de cada script (pulando os já aplicados), ou o agente precisa receber uma lista explícita de quais scripts a versão exige, em vez de apontar para a pasta inteira.

4.2 Do código do painel e do agente

CRÍTICO Uma atualização que falha some do radar em vez de ficar pendente

Assim que o agente decide baixar uma versão, ele já grava essa versão-alvo no campo VERSAO_NOVA — antes de qualquer autorização ou execução. Se a Fase 3 falhar e o estado virar ERRO, esse campo não é revertido. Na próxima consulta do agente, a API responde que não há nada mais novo, porque o valor enviado já é igual ao publicado.

O cliente fica permanentemente na versão antiga, mas tanto o agente quanto o painel param de sinalizar pendência — o único rastro é um log de erro antigo na aba Distribuição, que ninguém é obrigado a olhar de novo.

Onde: Worker.cs:50 (grava o alvo cedo demais) · Worker.cs:105 (não reverte no catch) · DatabaseService.cs:24-31 (lê o mesmo campo como "versão atual") · VersaoService.js:68-72 (compara os dois)

Cenário: BScript falha por disco cheio → estado ERRO, mas VERSAO_NOVA já é "2026.08.10" → próximo polling não detecta pendência → cliente nunca mais recebe a versão, sem alarme ativo.

Correção: Separar "versão instalada confirmada" de "versão-alvo em andamento", ou reverter VERSAO_NOVA ao valor anterior dentro do bloco catch de Worker.cs.

ALTO Injeção de BLOB no BEXE não é transacional nem revertida em falha

Cada executável é gravado no BEXE.fdb num UPDATE/INSERT isolado, sem transação explícita envolvendo o lote inteiro. Se o terceiro de cinco arquivos falhar, os dois primeiros já foram commitados. O bloco de rollback do Worker só sabe restaurar o JUNIOR.fdb a partir do backup — nunca toca o BEXE.fdb.

Onde: DatabaseService.cs:49-82 (InjetarNovosBinarios)

Cenário: Cinco EXEs, o quarto falha por lock de arquivo → três novos e dois antigos ficam em produção até a próxima tentativa bem-sucedida, se ela vier.

Correção: Envolver o laço numa única FbTransaction com commit ao final; incluir o BEXE no plano de rollback ou, no mínimo, alertar quando a Fase 4 falha parcialmente.

ALTO O campo script_url existe na API e no formulário, mas o agente nunca o lê

O painel aceita e devolve scriptUrl no contrato de versão. A classe de resposta do agente (UpdateResponse) não tem propriedade para esse campo, e o Worker sempre chama um caminho fixo (ATUALIZADOR_BSCRIPT_PATH) já presente no servidor do cliente.

Onde: ApiService.cs:82-88 (sem ScriptUrl) · Worker.cs:19 (caminho fixo) · VersaoService.js:93 (campo existe do lado do painel)

Cenário: Atualizar a lógica do próprio BScript ainda exige cópia manual em cada servidor — a promessa de "zero toque" não cobre o script mais crítico de todos.

Correção: Adicionar ScriptUrl a UpdateResponse e baixar/validar o script como qualquer outro pacote antes da Fase 3, ou remover o campo da API para não prometer o que o agente não faz.

MÉDIO MENSAGEM_LOG está na tabela de controle mas nunca é escrito

O DDL de SYS_ATUALIZACAO reserva um campo exatamente para isto, mas SetStatusAtualizacao só grava STATUS e VERSAO_NOVA. O detalhe do erro só sai pela chamada HTTP SendLog — que engole qualquer exceção de rede silenciosamente.

Onde: DatabaseService.cs:33-47 · ApiService.cs:68-79 (catch vazio)

Cenário: Se o servidor do cliente perder acesso à internet no momento da falha, não sobra nenhum rastro local do que aconteceu.

Correção: Gravar a mensagem de erro em MENSAGEM_LOG antes de tentar o envio HTTP, não depois/em vez dele.

MÉDIO Sem contenção após ERRO — mesmo depois de corrigido o achado crítico de versão

O laço principal roda a cada 10 segundos indefinidamente. Se o achado crítico de VERSAO_NOVA for corrigido e o agente voltar a tentar a mesma versão a cada ciclo, uma falha persistente dispara download, extração e tentativa de BScript a cada 10 segundos, sem limite de tentativas nem espera crescente.

Onde: Worker.cs:32-65

Correção: Backoff exponencial ou número máximo de tentativas antes de exigir intervenção manual.

BAIXO 7-Zip só é chamado para extensões conhecidas — a "extração universal" tem um filtro antes dela

O documento V4 descreve o 7-Zip como algo que ignora extensões e lê o cabeçalho binário. Isso é verdade sobre a ferramenta 7za.exe, mas o wrapper em C# só a invoca para arquivos que já terminam em .7z, .rar ou .zip.

Onde: ExtractionService.cs:26-28

Cenário: Um pacote com extensão inesperada seria ignorado silenciosamente, não extraído.

Correção: Impacto baixo hoje porque a API sempre envia extensões conhecidas — vale corrigir a redação do documento ou remover o filtro.

BAIXO Usuário e senha do Firebird viram uma única string de argumentos

"-user {user} -password {password}" é montado por interpolação e passado como string de argumentos para gfix/gbak. Uma senha com espaço ou aspas quebra o parsing do processo em vez de falhar com mensagem clara.

Onde: Worker.cs:72 (usado em 76, 78, 82, 84, 97, 99)

Cenário: Como o valor vem de configuração do operador (não de entrada de usuário final), o risco é de robustez, não de injeção explorável por terceiros.

Correção: Usar ProcessStartInfo.ArgumentList em vez de string formatada.

5. Ciclo de atualização — onde cada risco mora

As quatro fases descritas no documento técnico realmente existem no Worker.cs. Abaixo, em qual fase cada achado novo se materializa.

Fase	O que faz	Risco identificado
1 — Preparo invisível	Polling, download, SHA-256, extração, grava PENDENTE.	Grava a versão-alvo cedo demais — origem do achado crítico de versão-alvo.
2 — Decisão do usuário	Delphi lê PENDENTE, confirma, grava AUTORIZADO.	Não existe no repositório — nenhum arquivo .pas/.dpr encontrado.
3 — Execução crítica	GFIX → GBAK → BScript → GBAK → GFIX online.	BScript.exe provavelmente sem modo silencioso; nunca é distribuído via nuvem.
4 — Distribuição BEXE	Injeta cada EXE em commits separados.	Injeção não atômica; rollback da Fase 3 não cobre esta fase.

6. Documentos de arquitetura comparados ao código real

A especificação V4 e a apresentação executiva descrevem a mesma arquitetura mais antiga (MD5, sem estado PROCESSANDO). O documento técnico v1.0 é o único que já reflete decisões visíveis no código atual.

Afirmção	V4 / Executivo	Doc. técnico v1.0	Código real
Verificação de integridade do pacote	MD5	SHA-256	SHA-256 confirmado
Estados de SYS_ATUALIZACAO	4 estados, sem PROCESSANDO	5 estados incl. PROCESSANDO	5 estados no código
Distribuição do BScript.exe via nuvem	Sim, script_url	Não detalha	Campo ignorado pelo agente
Erro "não estrutural" do BScript	—	—	Não existe — qualquer erro aborta
Token de autenticação do agente	Não especifica	X-Agent-Token	Implementado (token compartilhado entre clientes)
Atomicidade da injeção BEXE	Não trata	Não trata	Confirmado: não é atômica
Cadência de polling	"Diariamente"	"Intervalos regulares"	10s no código; produção não confirmada
Schema real do BEXE.fdb	Assume UPDATE...BLOB	Lista como pendência	Tabela real é EXECUTAVEIS, não VERSOES_EXE — código ainda não corrigido
BScript.exe suporta CLI/silencioso	Assume que sim	Assume que sim	Nenhuma string de parsing de linha de comando no binário real
Integração Delphi (Fase 2)	Descrita	Lista como pendência	Nenhum arquivo .pas no repositório

7. Pendências que os próprios documentos já assumem

O documento técnico v1.0 lista quatro pontos em sua própria seção de pendências. A auditoria contra os arquivos reais resolveu um deles e piorou a leitura de outro.

- **Schema do BEXE.fdb — RESOLVIDO.** Tabela real é EXECUTAVEIS, confirmada abrindo o BEXE.FDB real. Falta corrigir o código para usá-la. O schema do JUNIOR.fdb (tabela SYS_ATUALIZACAO em si) segue não confirmado — nenhuma cópia desse banco estava disponível.
- **Comando silencioso real do BScript — PIOROU.** Não é mais "ainda não testado": a engenharia reversa não encontrou evidência de suporte a linha de comando, e encontrou fortes evidências de interface puramente interativa. Já é risco ativo, não pendência de confirmação.
- **Procedimento de restauração** — o comando `gbak -c -replace_database` existe no código de rollback, mas nunca foi exercitado contra uma cópia real do Firebird 2.5 do cliente.
- **Integração Delphi** — não há qualquer arquivo `.pas/.dpr` no repositório; a Fase 2 inteira ainda precisa ser escrita no ERP real.

8. Prioridades antes de um piloto

#	Prioridade	Por quê
1	Resolver a questão do BScript.exe antes de qualquer outra coisa	Se ele de fato não tem modo silencioso, automatizá-lo pode travar o servidor do cliente inteiro (banco single-user, ninguém consegue vender) até intervenção manual. Nenhum piloto deveria rodar contra a Fase 3 atual sem essa confirmação.
2	Corrigir o campo de versão-alvo em falha	É o achado capaz de fazer um cliente parar de receber atualizações sem que ninguém perceba — silenciosamente, sem exigir coincidência rara.
3	Reescrever a injeção do BEXE contra o schema real e cobri-la no rollback	O código aponta para uma tabela que não existe no arquivo real — hoje falharia de imediato. Depois de corrigido, ainda falta transação única e cobertura no rollback.
4	Decidir como os 1026 scripts serão aplicados por cliente	Sem lista de "quais scripts esta versão exige" ou tabela de controle automática, rodar a pasta inteira em cliente que não parte de banco zerado gera erros a cada tentativa.
5	Decidir o destino do campo script_url	Ou o agente passa a baixá-lo e a promessa de zero-toque vale para o BScript também, ou o campo sai da API.
6	Confirmar os pontos que restam do documento técnico	Schema real do JUNIOR.fdb, teste de restauração do backup, implementação da Fase 2 em Delphi — nenhum piloto aponta a ponta é possível sem eles.
7	Adicionar contenção (backoff) ao laço de 10 segundos	Menos urgente, mas evita ruído constante de disco/rede assim que o item 2 for corrigido e o retry voltar a acontecer de fato.

9. Conclusão

O projeto é mais real do que qualquer um dos três documentos de arquitetura deixa transparecer: o agente C#/.NET existe, compila, e implementa a sequência GFIX → GBAK → BScript → GBAK → injeção de BLOB → GFIX exatamente como descrita no documento técnico v1.0. Catorze comportamentos de segurança e orquestração foram confirmados linha a linha.

Ao mesmo tempo, ler o código e os binários reais deslocou o risco principal do projeto. O medo inicial — uma heurística de "erro não estrutural" do BScript mascarando falha como sucesso — não existe no código: qualquer código de saída diferente de zero já aborta. O risco real e silencioso é outro: o campo de versão-alvo nunca sendo revertido em falha, e a possibilidade concreta de que o BScript.exe de produção não tenha modo silencioso algum — o que pode travar o servidor do cliente, não apenas falhar.

Recomendação: tratar os itens 1 e 2 da seção 8 como bloqueadores absolutos de qualquer piloto. Os demais achados são reais e devem ser corrigidos, mas não têm o mesmo potencial de causar indisponibilidade total no servidor do cliente ou perda silenciosa de atualização.